

FitTrack AI

Personalized Fitness Recommendations Powered by AI

Phase 5 – Project Development

Project type: AI-powered fitness tracking and recommendation REST API / backend

Stack: Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · REST APIs · Thunder Client / Postman

1. Project Setup

1.1 Node.js Environment

FitTrack AI runs on Node.js (LTS version), which executes JavaScript on the server. The installed versions can be checked with `node -v` and `npm -v` before starting.

1.2 npm Package Management

npm creates and manages the project. Running `npm init` generates `package.json`, which records the project details, the dependency list and the run scripts. Dependencies are installed with `npm install`, and they are stored in the `node_modules` folder, which is excluded from Git through `.gitignore`.

1.3 Required npm Dependencies

<code>express</code>	Web framework for the server, routes and middleware.
<code>mongoose</code>	Connects to MongoDB Atlas and defines the User and Workout schemas.
<code>jsonwebtoken</code>	Creates and verifies JWT tokens for authentication.
<code>bcryptjs</code>	Hashes and compares passwords (bcrypt.js).

<code>cors</code>	Enables cross-origin requests to the API.
<code>morgan</code>	Logs each HTTP request to the console during development.
<code>dotenv</code>	Loads environment variables from the <code>.env</code> file.
Google Gemini SDK (for example <code>@google/generative-ai</code>)	Calls the Google Gemini API from the Gemini AI service. Use the package that appears in your <code>package.json</code> .
<code>nodemon</code> (development dependency)	Restarts the server automatically when files change.

1.5 Environment Variables

Environment variables keep configuration and secrets outside the source code. They are stored in a `.env` file, loaded with `dotenv`, and read in code through `process.env`. The `.env` file is listed in `.gitignore` so it is never pushed to GitHub.

<code>PORT</code>	<code>5000</code>	Port on which the server listens.
<code>MONGO_URI</code>	<code><your-mongodb-atlas-connection-string></code>	Connection string to the MongoDB Atlas cluster. It contains database credentials.
<code>JWT_SECRET</code>	<code><long-random-secret></code>	Key used to sign and verify JWT tokens.
<code>GEMINI_API_KEY</code>	<code><your-gemini-api-key></code>	Authorizes requests to the Google Gemini API.

Why environment variables are used for sensitive values:

- **MongoDB connection string** contains the database username and password. If it leaks, anyone can read or change the data.
- **JWT secret** signs every token. Anyone who knows it can create valid tokens and impersonate any user.
- **Gemini API key** is tied to the owner's account and usage quota. A leaked key can be misused and consume that quota.

Keeping these out of the code also lets the same code run with different values on different machines, and a compromised value can be replaced without editing the code.

1.6 Server Configuration

`server.js` is the entry point. It loads the environment variables, connects to MongoDB Atlas, and starts the Express application listening on the configured port. The server only starts accepting requests after the database connection succeeds, and if the connection fails the error is logged and the process exits.

1.7 MongoDB Connection Configuration

The connection logic lives in `config/db.js`. It calls Mongoose's `connect()` with the connection string from `MONGO_URI`. Before this works, a MongoDB Atlas cluster is created, a database user is added, and the developer's IP address is allowed in Atlas network access. On success a confirmation message is printed; on failure the error is shown.

```
npm install # install all dependencies
```

Once the server is running, the API is tested by sending requests from Thunder Client or Postman to the local server address and port.

1.9 Setup Summary Table

Runtime	Node.js (LTS)	Runs the JavaScript backend on the server.
Package management	npm, <code>package.json</code>	Installs dependencies and defines run scripts.
Web framework	Express.js	Handles HTTP requests, routing and middleware.

Folder structure	routes, middleware, controllers, services, models, config	Separates responsibilities for a maintainable codebase.
Environment variables	<code>.env</code> with dotenv	Keeps the MongoDB string, JWT secret and Gemini key out of the code.
Server configuration	<code>server.js</code> , PORT variable	Loads configuration, connects to the database, starts listening.
Database connection	Mongoose + MongoDB Atlas (<code>config/db.js</code>)	Connects the application to the cloud database.
Development execution	nodemon (<code>npm run dev</code>)	Runs the server with automatic restart.
API testing tool	Thunder Client / Postman	Sends requests and checks JSON responses.

2. Backend Development

2.1 Express Application

The Express application (`app.js`) is the core of the backend. It registers the global middleware, mounts the route files, and registers the error handler last. It is kept separate from `server.js`, so the application setup is independent of starting the server.

2.2 Middleware

JSON middleware (<code>express.json()</code>)	Parses JSON request bodies so controllers can read <code>req.body</code> .
---	--

CORS (<code>cors()</code>)	Controls which origins may call the API.
Morgan	Logs the method, URL, status code and response time of every request.
Authentication middleware	Reads the Bearer token, verifies it with the JWT secret, loads the user and attaches it to <code>req.user</code> . Invalid or missing tokens receive a 401 response.
Centralized error handler	Receives every error passed with <code>next(error)</code> and returns a consistent JSON error response with the correct status code.

2.3 Routes, Controllers and Services

- **Routes** connect a URL and HTTP method to a controller function and attach the authentication middleware to protected endpoints. They are grouped into authentication, workout and AI routes.
- **Controllers** handle the request and response: they read the input, call the service and send the standard JSON response.
- **Services** contain the business logic, such as password hashing, token creation, workout queries limited to the logged-in user, and the Gemini AI service that builds prompts and calls the Google Gemini API.
- **Models** define the schema for the User and Workout collections and communicate with MongoDB Atlas through Mongoose.

2.4 Modular Structure and Request Lifecycle

A request enters through the Express application, passes the global middleware, matches a route, passes through the authentication middleware if the route is protected, reaches a controller, and is handled by a service that uses the Mongoose models (MongoDB Atlas) or the Gemini AI service (Google Gemini). Errors at any step go to the centralized error handler. Because each layer only talks to the next one, a change in one place (for example, changing the AI prompt) does not affect the rest of the code.

FitTrack AI – Backend Development Architecture
 Serverserver.js · load .env · connect DB · listen on PORT Express Applicationapp.js · global middleware · mount routes RoutesAuth routes · Workout routes · AI routes MiddlewareAuthentication (JWT verification) · req.user ControllersRead input · call services · JSON response Services / ModelsBusiness logic · Mongoose models · Gemini AI service MongoDB AtlasUsers and Workouts Google Gemini AIRecommendation / Insight GLOBAL MIDDLEWARE express.json() cors() morgan (logging) .env variablesPORT · MONGO_URI · JWT_SECRET Centralized Error Handler next(error) → JSON error

response Legend Server / app Routes Middleware Controllers / services Database / errors AI Solid arrows: request flow. Dashed red arrows: errors passed to the centralized error handler. Figure 1: FitTrack AI – Backend Development Architecture

3. Database Development

3.1 MongoDB Atlas and Mongoose

MongoDB Atlas is the cloud database that stores all application data in two collections: **users** and **workouts**. Mongoose is the object modeling library that connects Node.js to MongoDB. Each collection is defined by a Mongoose schema, which sets the fields, their data types and their validation rules, and Mongoose creates the model used by the services to read and write data. Both schemas use the timestamps option, so `createdAt` and `updatedAt` are added and maintained automatically.

3.2 User Model

<code>_id</code>	ObjectId	Primary key, generated automatically by MongoDB.
<code>name</code>	String	Required. The user's name.
<code>email</code>	String	Required and unique, so two accounts cannot share an email address. Used to log in.
<code>password</code>	String	Required. Stored only as a bcrypt hash, never as plain text.
<code>createdAt</code>	Date	Set automatically when the user is created.
<code>updatedAt</code>	Date	Updated automatically on every change.

3.3 Workout Model

<code>_id</code>	ObjectId	Primary key, generated automatically by MongoDB.
<code>workoutName</code>	String	Required. Name of the workout.
<code>category</code>	String	Required. Validated against the allowed workout categories defined in the schema (for example, using a Mongoose <code>enum</code>). <i>List your project's actual categories here.</i>
<code>duration</code>	Number	Required. Length of the workout (in minutes).
<code>caloriesBurned</code>	Number	Number of calories burned in the workout.
<code>workoutDate</code>	Date	Date on which the workout took place.
<code>user</code>	ObjectId (ref: User)	Required. Foreign key that stores the <code>_id</code> of the user who owns the workout.
<code>createdAt</code>	Date	Set automatically.
<code>updatedAt</code>	Date	Updated automatically.

3.4 Validation, Uniqueness and Password Storage

- **Required fields:** Mongoose rejects a document if a required field is missing and returns a validation error, which the centralized error handler converts to a JSON response.
- **Unique email:** the unique index on `email` is enforced by MongoDB, so duplicate registrations fail.
- **Password storage:** the password is hashed with `bcrypt.js` (with a salt) before saving. At login, the entered password is compared with the stored hash using `bcrypt.js`; the original password cannot be recovered from the hash.

- **Category validation:** only categories from the allowed list are accepted for a workout.

3.5 User-Workout Relationship

The relationship is one-to-many: one User can have many Workouts, and each Workout belongs to exactly one User. The Workout model's `user` field stores the User's `_id` with `ref: "User"`. Workout queries filter by this field, so users see and modify only their own workouts. Mongoose `populate()` can be used to replace the reference with the user document when needed.

FitTrack AI – Database Structure Diagram MongoDB Atlas · Mongoose models User
`_id` : ObjectIdPK name : Stringrequired email : Stringrequired · unique password : Stringrequired · bcrypt hash createdAt : Dateauto updatedAt : Dateauto 1 has many N Workout
`_id` : ObjectIdPK workoutName : Stringrequired category : Stringrequired · allowed values duration : Numberrequired caloriesBurned : Number workoutDate : Date user : ObjectIdFK · ref: User createdAt : Dateauto updatedAt : Dateauto Key database rules · Each model uses timestamps, so createdAt / updatedAt are automatic. · User.email is unique: duplicate registrations are rejected. · User.password is saved only as a bcrypt hash. · Workout.category must be one of the allowed categories. · Workout.user stores the owner's User._id, so queries can be limited to the logged-in user. · One User has many Workouts; each Workout has one User. Legend PK = Primary Key FK = Foreign Key / reference to User._id 1 — N = one-to-many relationship Figure 2: FitTrack AI – Database Structure Diagram